ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Nhập môn An toàn thông tin

Chương 9. **An toàn bộ nhớ**

Nội dung

- Cấu trúc vùng nhớ của tiến trình
- Lỗ hổng tràn bộ đệm (Buffer Overflow)
- Lỗ hổng tràn số nguyên
- Lỗ hổng xâu định dạng

1. Bộ nhớ của tiến trình

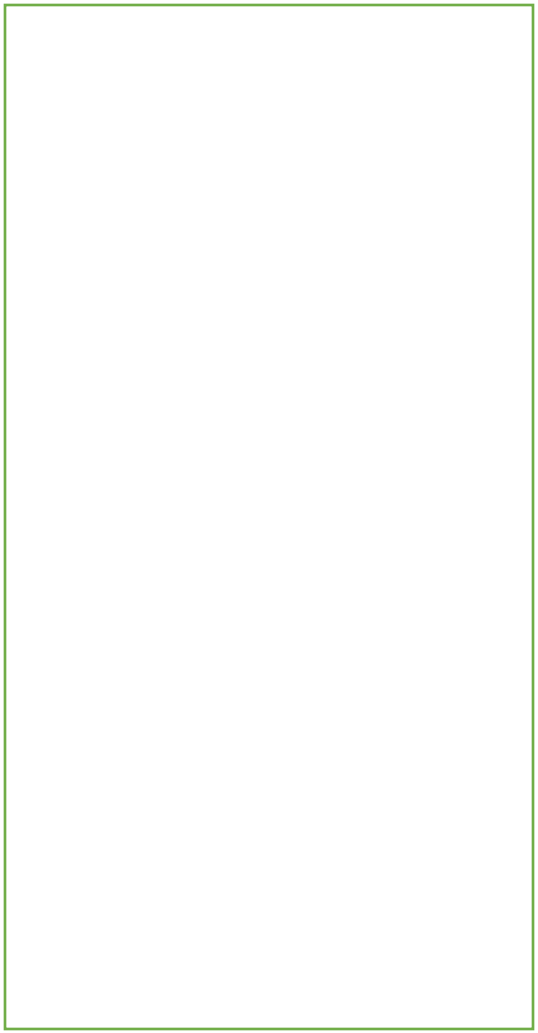
Tiến trình là gì?

- Là chương trình đang được thực hiện
- Các tài nguyên tối thiểu của tiến trình:
  - Vùng nhớ được cấp phát
  - Con trỏ lệnh(Program Counter)
  - Các thanh ghi của CPU
- Khối điều khiển tiến trình(Process Control Block-PCB): Cấu trúc chứa thông tin của tiến trình

Bộ nhớ của tiến trình(Linux 32-bit)

0xffffffff

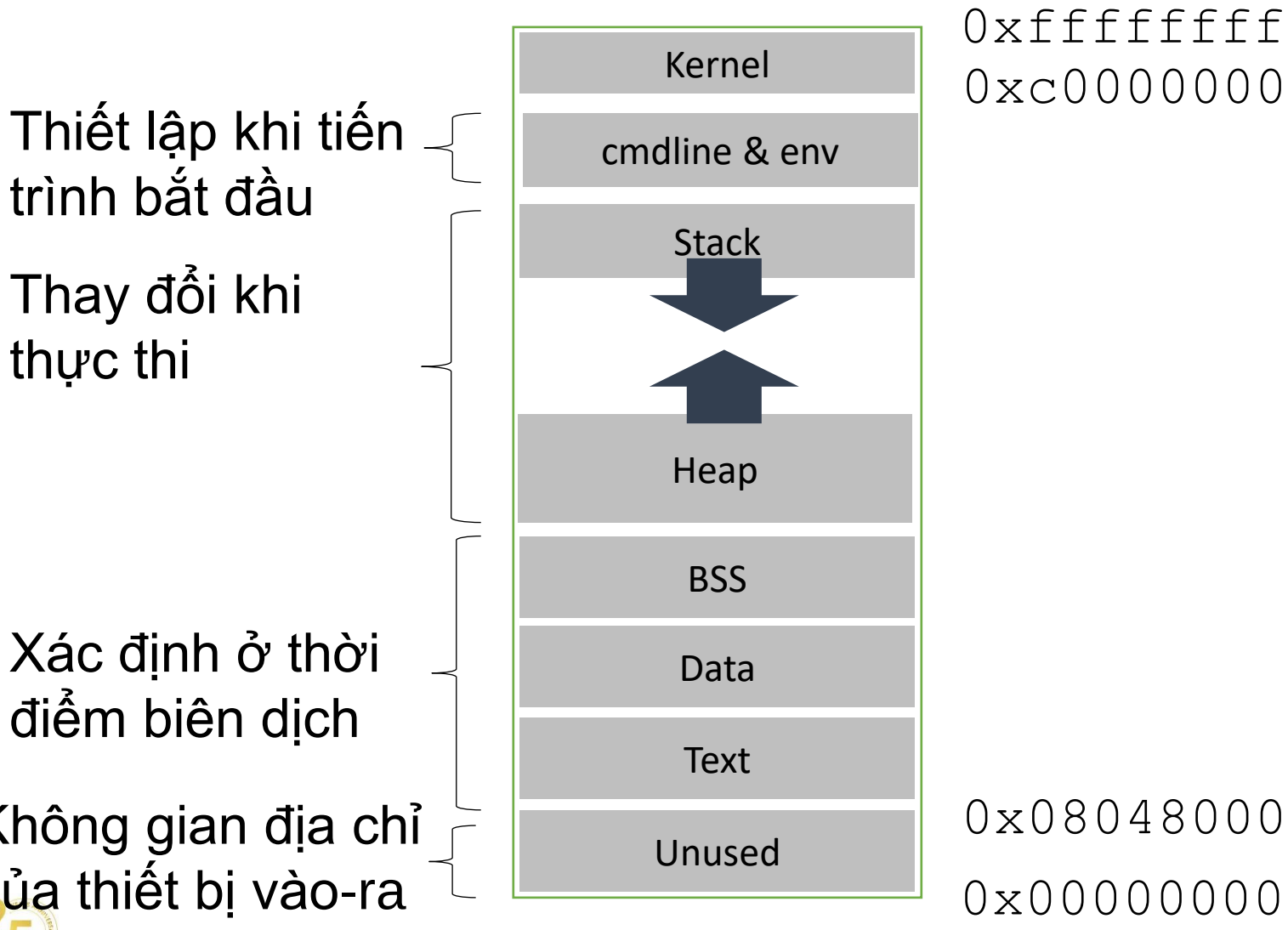
Tiến trình coi bộ nhớ thuộc toàn bộ sở hữu của nó



Thực tế đây là bộ nhớ ảo với địa chỉ ảo, sẽ được HĐH/CPU ánh xạ sang địa chỉ vật lý

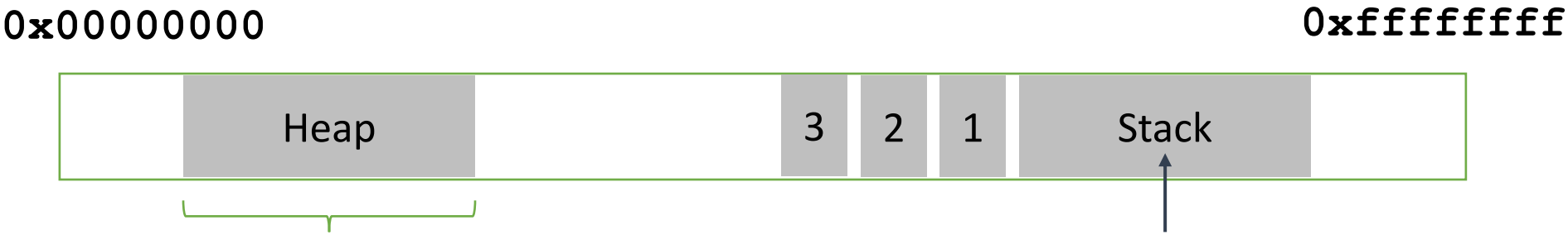
0x00000000

Bộ nhớ của tiến trình(Linux 32-bit)



Vùng nhớ stack và heap

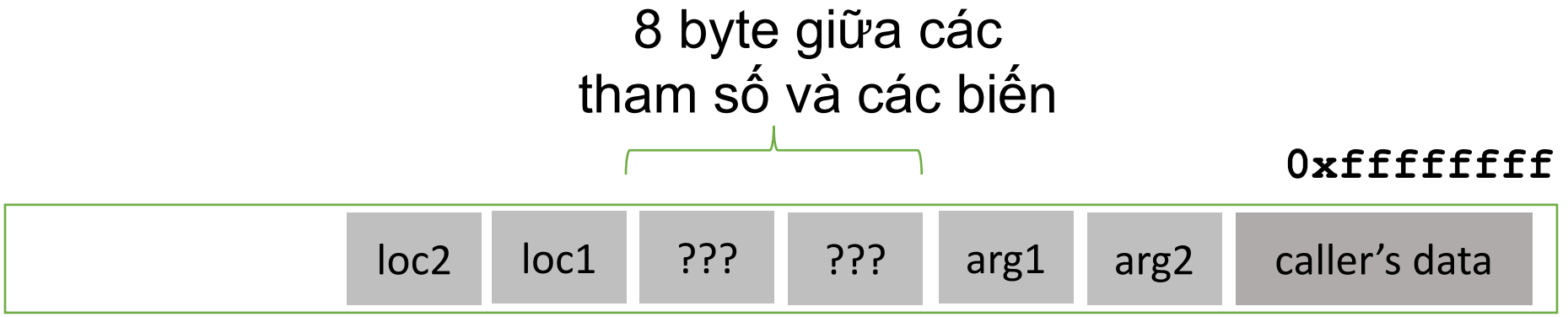
Trình biên dịch cung cấp các hàm làm thay đổi kích thước vùng nhớ stack khi thực thi chương trình



Được quản lý trong tiến trình bởi các hàm cấp phát bộ nhớ động (malloc, calloc)

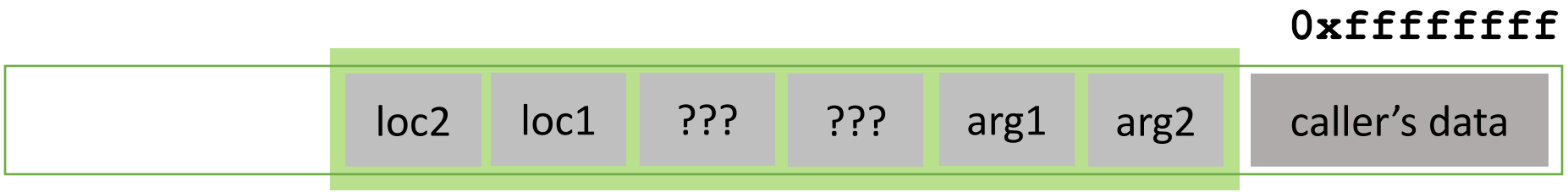
Stack - Thực hiện lời gọi hàm

```
void func(char *arg1, int arg2)
{
    char loc1[4];
    int loc2;
}
```



Stack frame

```
void func(char *arg1, int arg2)
{
    char loc1[4];
    int loc2;
}
```



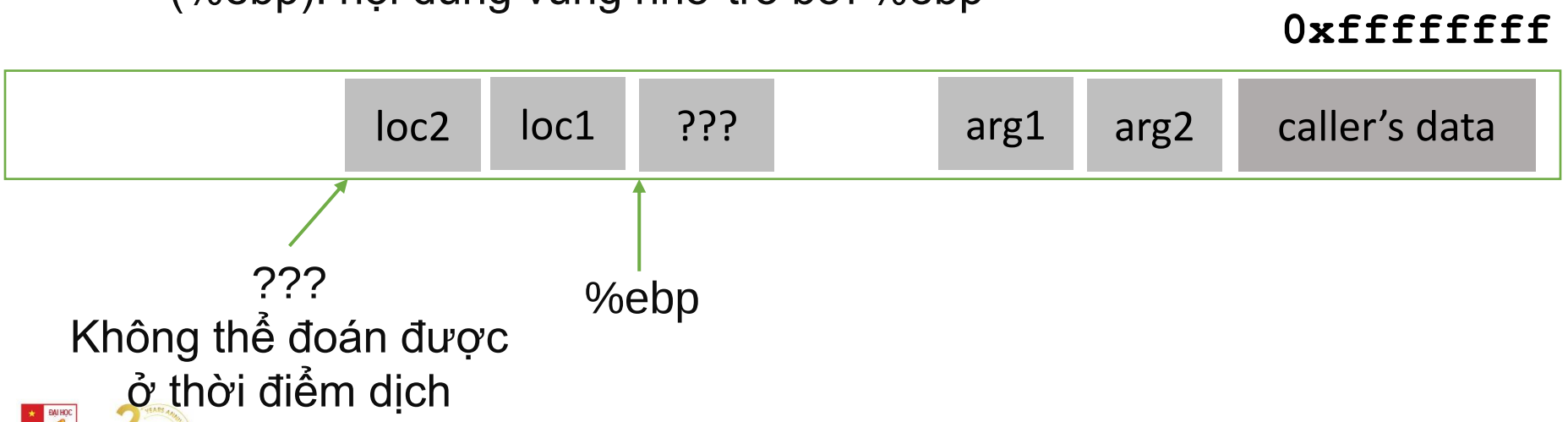
Stack frame: Một phần của vùng nhớ stack tương ứng với lời gọi của một hàm

Stack frame

```
void func(char *arg1, int arg2)
{
    char loc1[4];
    int loc2;
    loc2++;
}
```

Q: loc2 nằm ở đâu?
A: -8(%ebp)

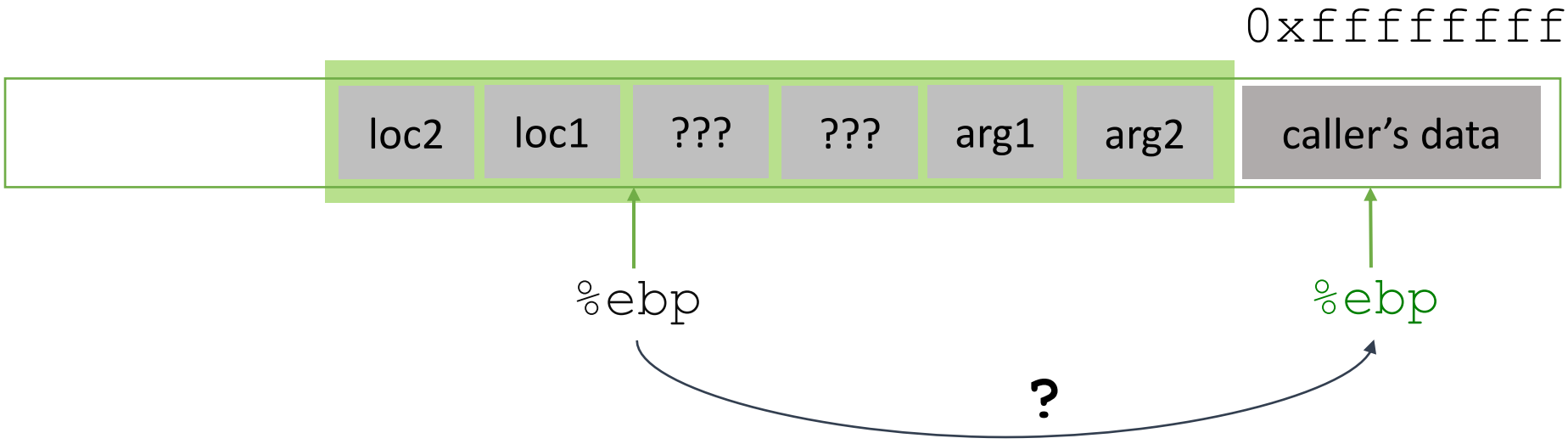
- %ebp: con trỏ frame.
- (%ebp): nội dung vùng nhớ trỏ bởi %ebp



Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

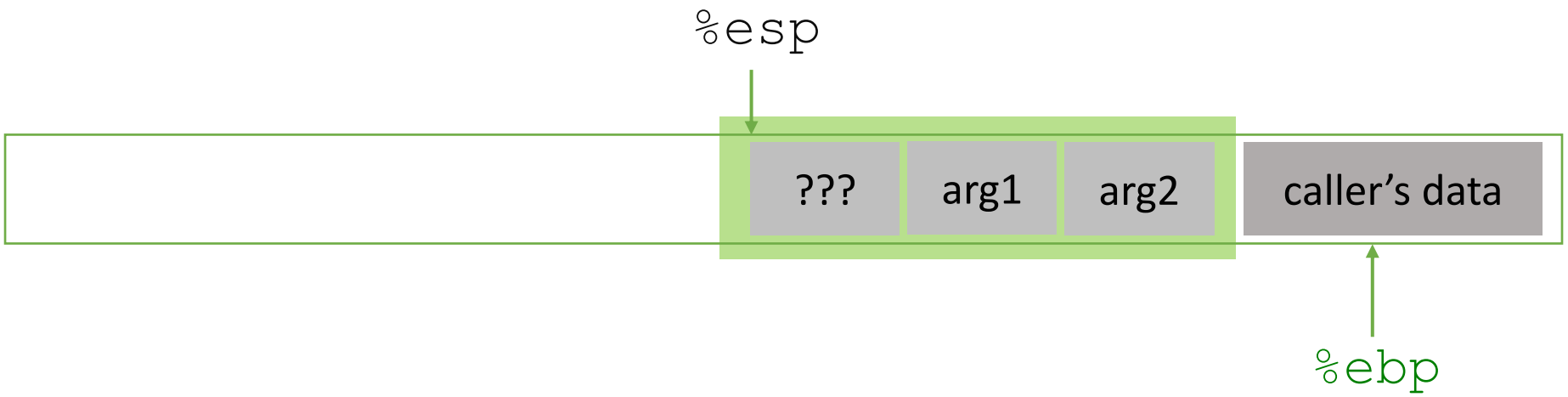
Q: Làm cách nào để khôi phục %ebp của hàm gọi



Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

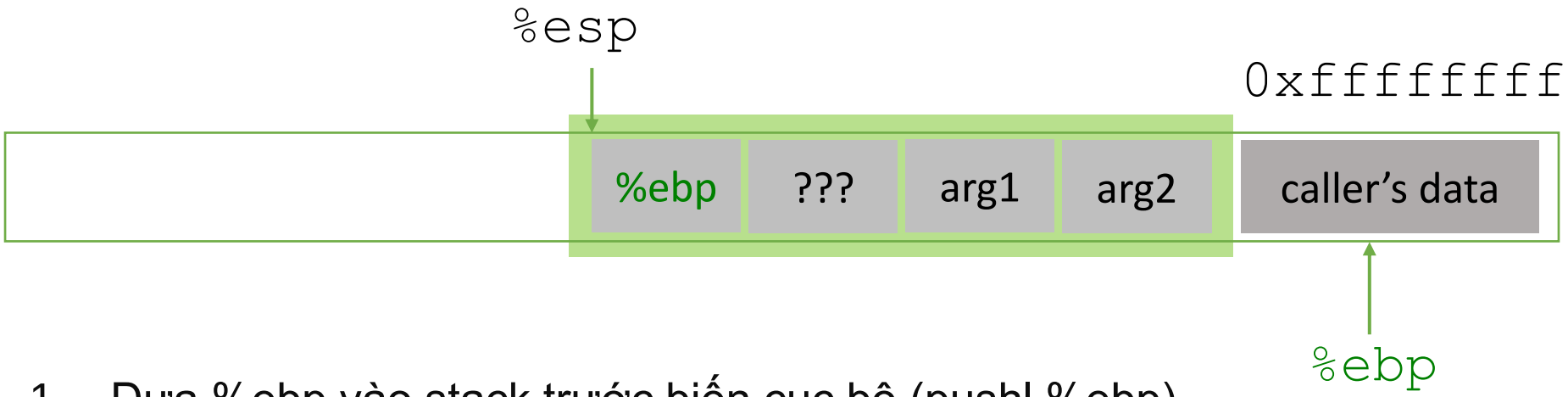
Q: Làm cách nào để khôi phục %ebp của hàm gọi



Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

Q: Làm cách nào để khôi phục %ebp của hàm gọi

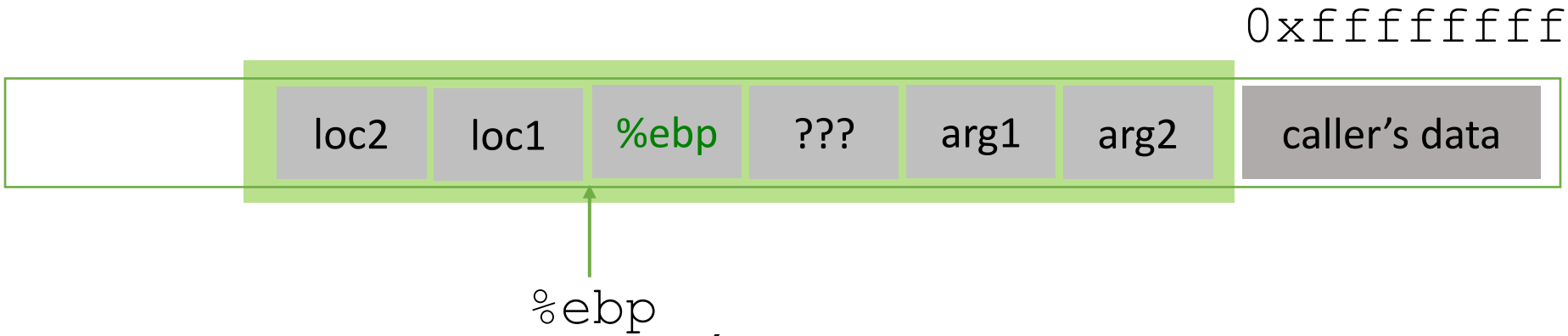


1. Đưa %ebp vào stack trước biến cục bộ (pushl %ebp)

Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

Q: Làm cách nào để khôi phục %ebp của hàm gọi

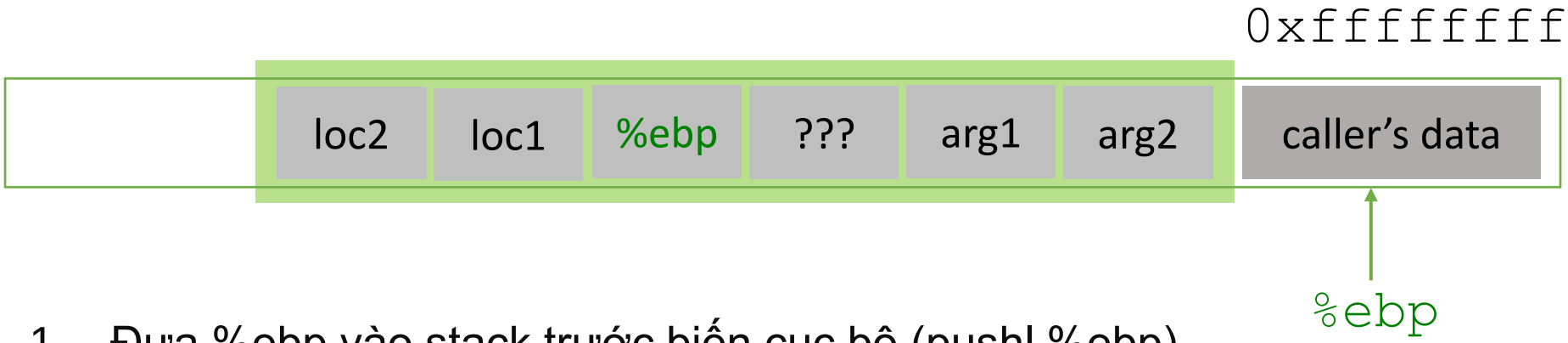


1. Đưa %ebp vào stack trước biến cục bộ (pushl %ebp)
2. Thiết lập %ebp bằng với %esp (movl %esp %ebp)

Stack – Trả về từ hàm

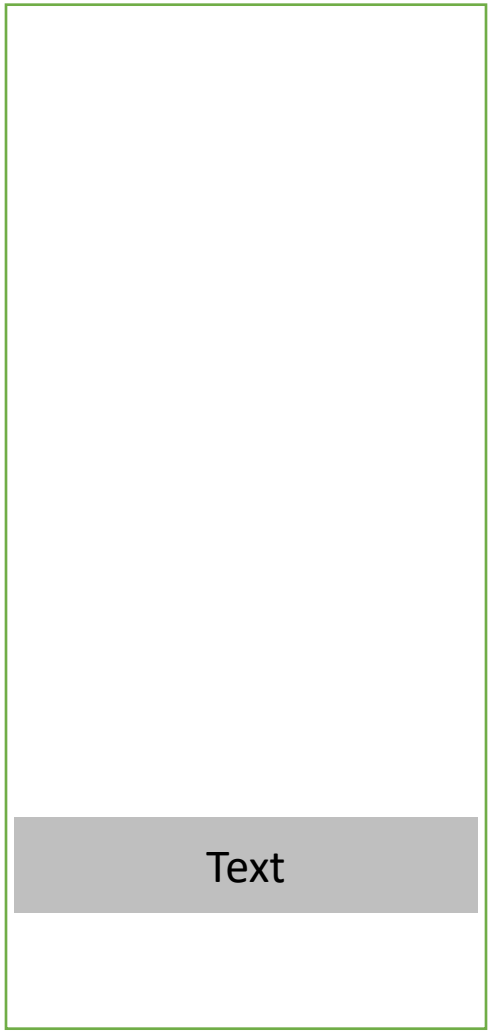
```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

Q: Làm cách nào để thực thi tiếp lệnh sau khi hàm trả về



1. Đưa %ebp vào stack trước biến cục bộ (pushl %ebp)
2. Thiết lập %ebp bằng với %esp (movl %esp %ebp)
3. Khi hàm trả về, thiết lập %ebp bằng (%ebp) (movl (%ebp) %ebp)

Con trỏ lệnh - %eip




```
...
0x5bf mov %esp,%ebp
0x5be push %ebp
...
```

```
...
0x4a7 mov $0x0,%eax
0x4a2 call <func>
0x49b movl $0x804..,(%esp)
0x493 movl $0xa,0x4(%esp)
...
```

← %eip

Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

Q: Làm cách nào để khôi phục %ebp của hàm gọi

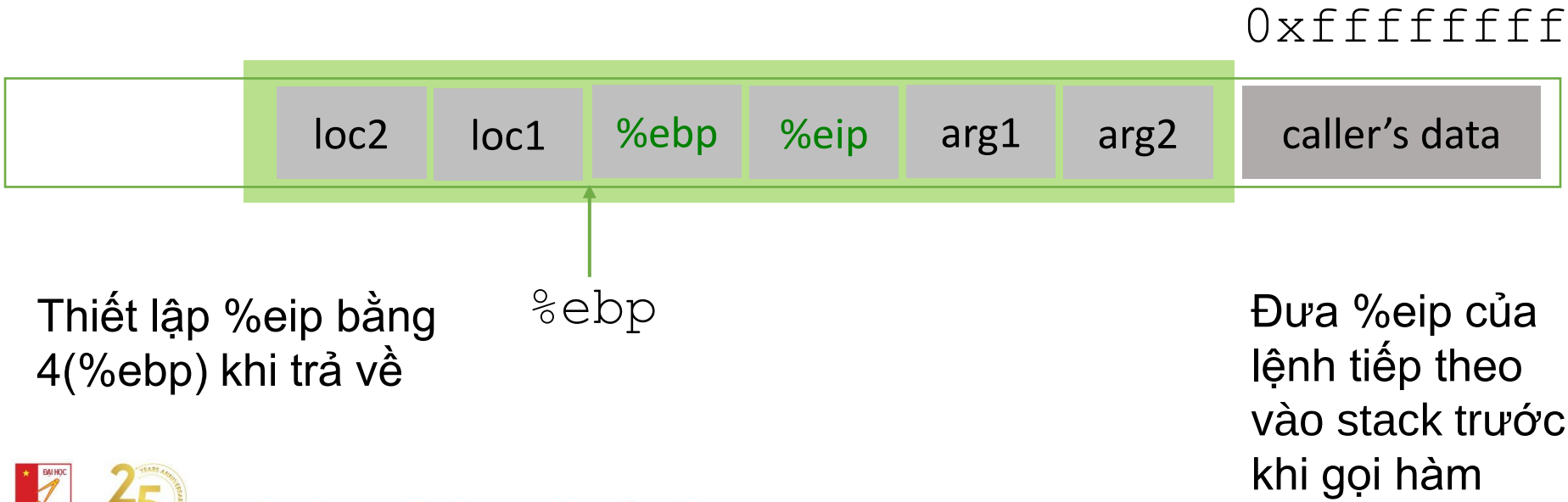


Đưa %eip của lệnh tiếp theo vào stack trước khi gọi hàm

Stack – Trả về từ hàm

```
int main()
{
    ...
    func("Hey", 10);
    ...
}
```

Q: Làm cách nào để khôi phục %ebp của hàm gọi



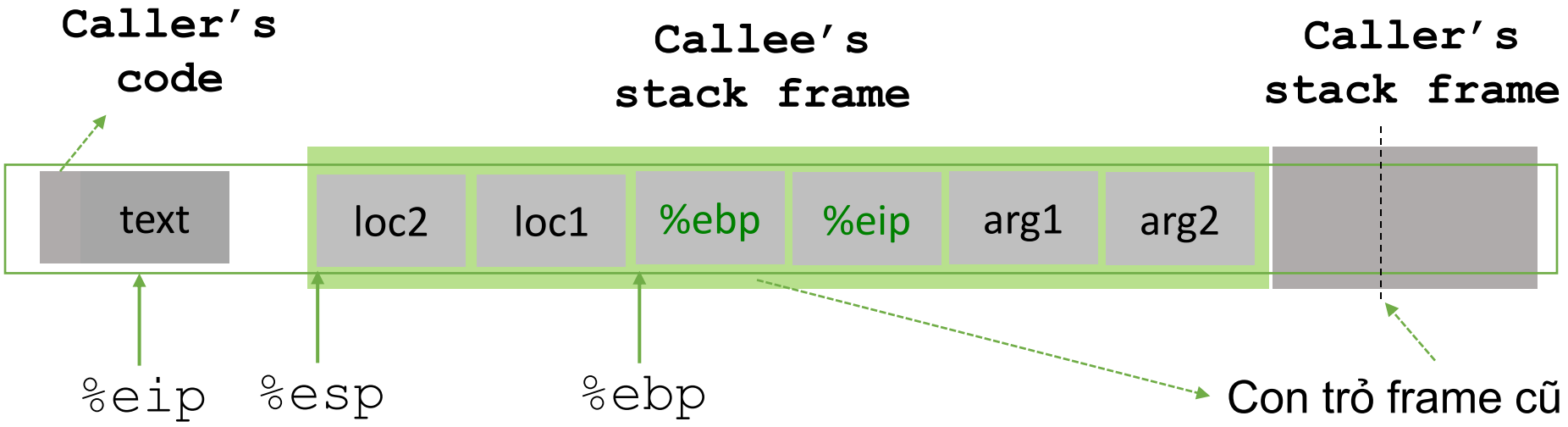
Stack – Trả về từ hàm

Trong C

```
return;
```

Mã assembly sau khi dịch

```
leave:    mov %ebp %esp
          pop %ebp
ret:      pop %eip
```



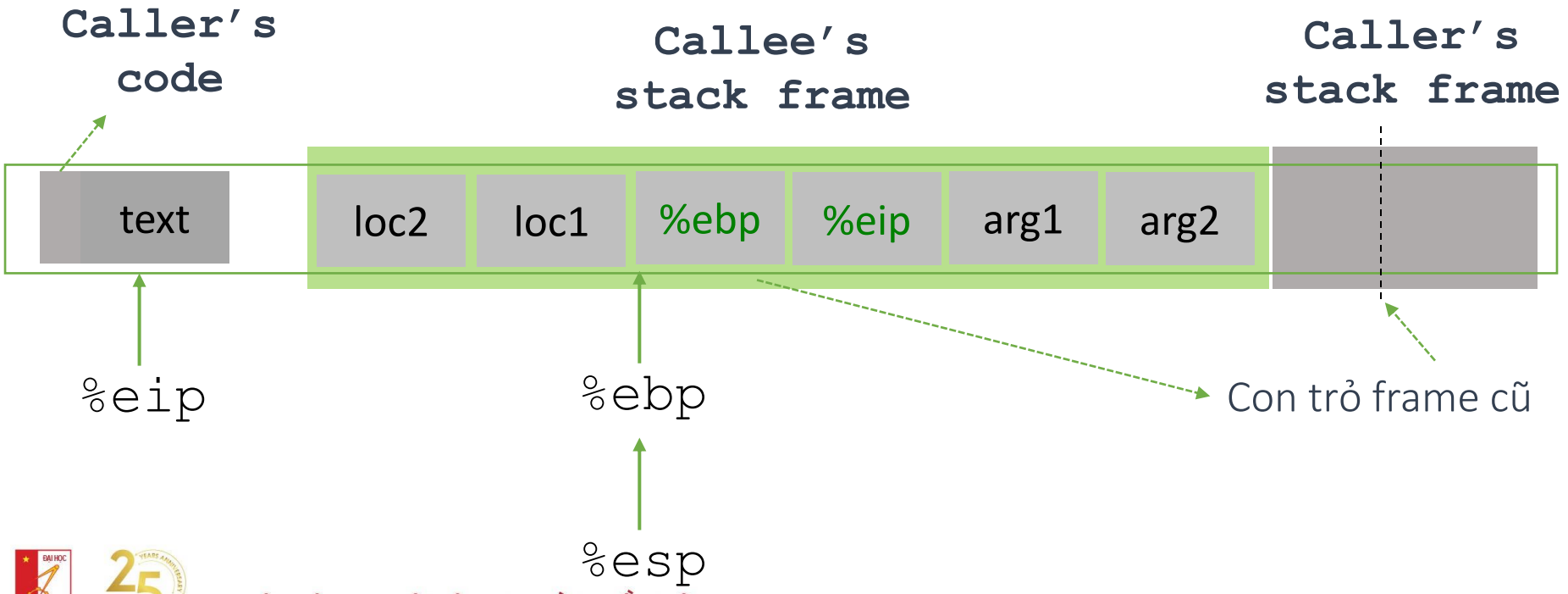
Stack – Trả về từ hàm

Trong C

```
return;
```

Mã assembly sau khi dịch

```
leave:  mov %ebp %esp
        pop %ebp
ret:    pop %eip
```



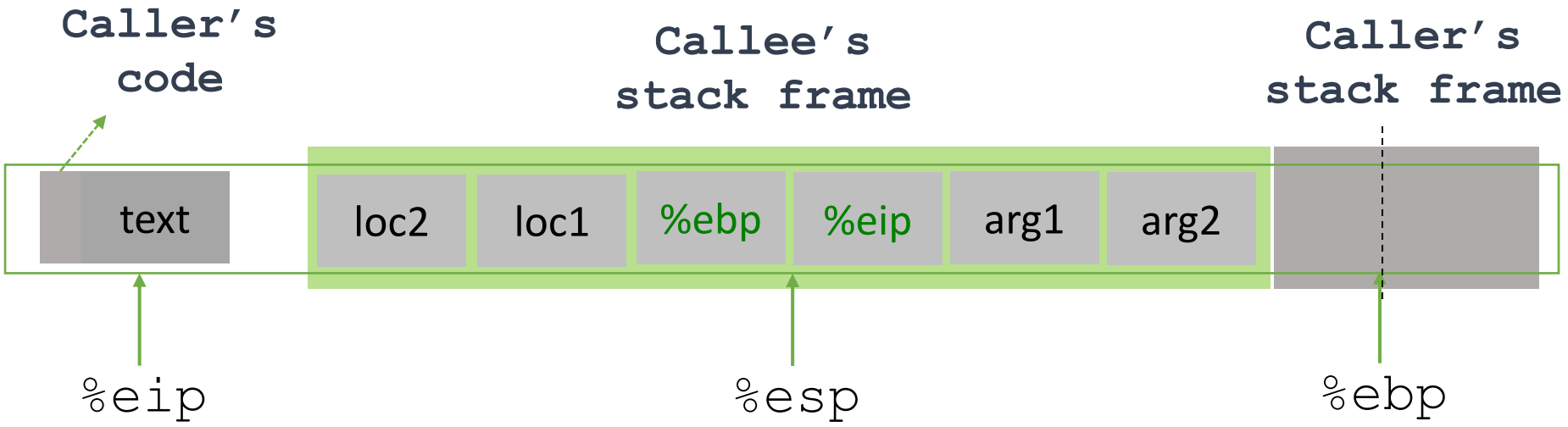
Stack – Trả về từ hàm

Trong C

```
return;
```

Mã assembly sau khi dịch

```
leave:    mov %ebp %esp
          pop %ebp
ret:      pop %eip
```



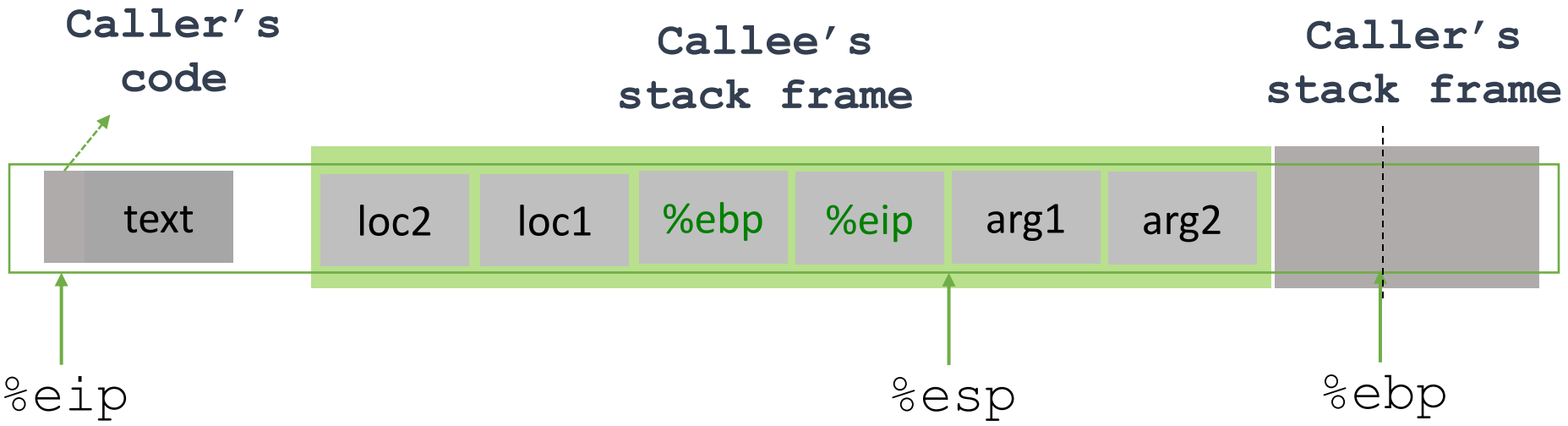
Stack – Trả về từ hàm

Trong C

```
return;
```

Mã assembly sau khi dịch

```
leave:    mov %ebp %esp
          pop %ebp
ret:      pop %eip
```



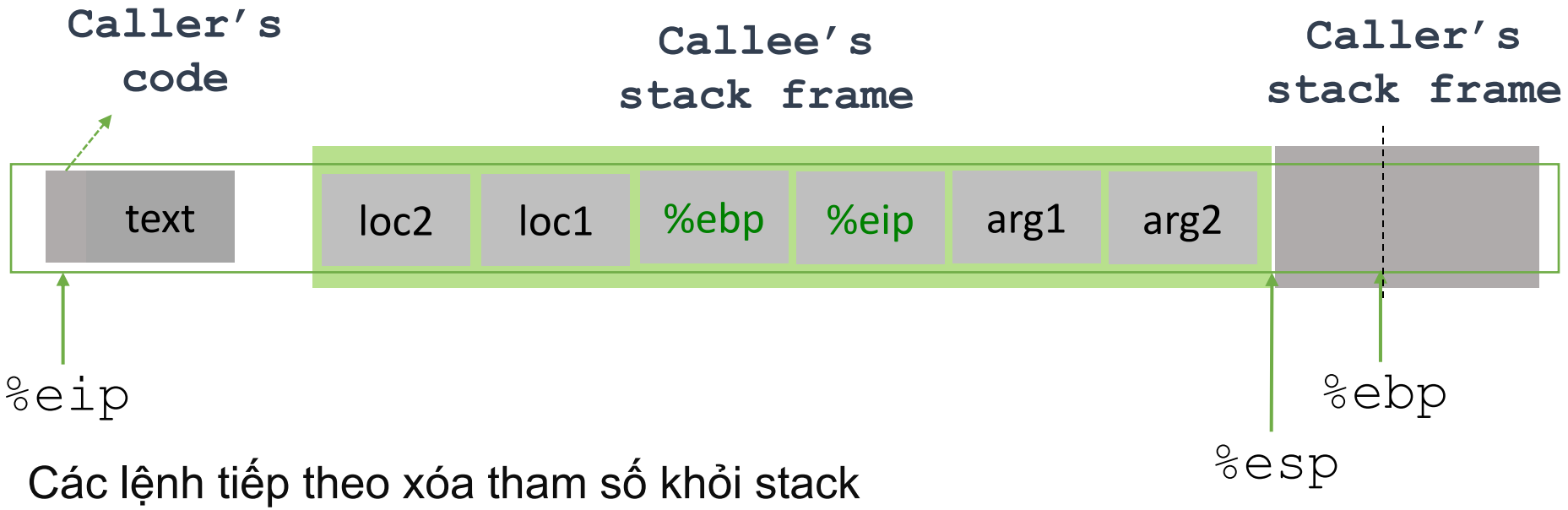
Stack – Trả về từ hàm

Trong C

```
return;
```

Mã assembly sau khi dịch

```
leave:    mov %ebp %esp
          pop %ebp
ret:      pop %eip
```



Các lệnh tiếp theo xóa tham số khỏi stack

Tổng kết

Hàm gọi(trước khi gọi):

1. Đẩy các tham số vào stack theo thứ tự ngược
2. Đẩy địa chỉ trả về vào stack, ví dụ %eip+2
3. Nhảy tới địa chỉ của hàm được gọi

Hàm được gọi:

4. Đẩy %ebp cũ vào stack
5. Thiết lập %ebp tới đỉnh của stack
6. Đẩy các biến cục bộ vào stack truy cập theo độ lệch từ %ebp

Hàm được gọi trả về:

7. Thiết lập lại %ebp cũ
8. Nhảy tới địa chỉ trả về

Hàm gọi:

9. Xóa các tham số khỏi stack

2. Lỗ hổng tràn bộ đệm

Khái niệm

- Bộ đệm (Buffer): tập hợp liên tiếp các phần tử kiểu dữ liệu xác định
  - VD: Trong ngôn ngữ C, xâu là bộ đệm của các ký tự
- Tràn bộ đệm (Buffer Overflow): Đưa dữ liệu vào bộ đệm nhiều hơn khả năng chứa của nó
- Lỗ hổng tràn bộ đệm: Không kiểm soát kích thước dữ liệu đầu vào.
- Tấn công tràn bộ đệm: Phần dữ liệu tràn ra khỏi bộ đệm làm thay đổi luồng thực thi của tiến trình.

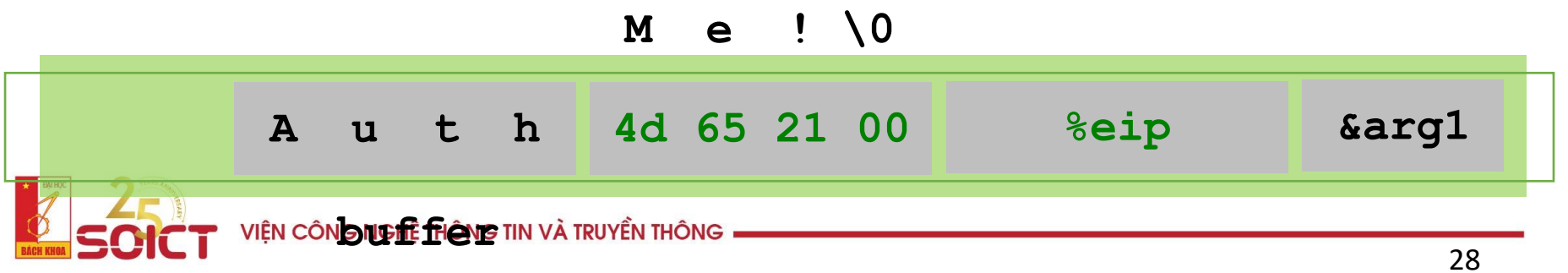
Ví dụ về tràn bộ đệm

```
void func(char *arg1)
{
    char buffer[4];
    strcpy(buffer, arg1);
    return;
}
int main()
{
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```



Ví dụ về tràn bộ đệm

```
void func(char *arg1)
{
    char buffer[4];
    strcpy(buffer, arg1);
    return;
}
int main()
{
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```

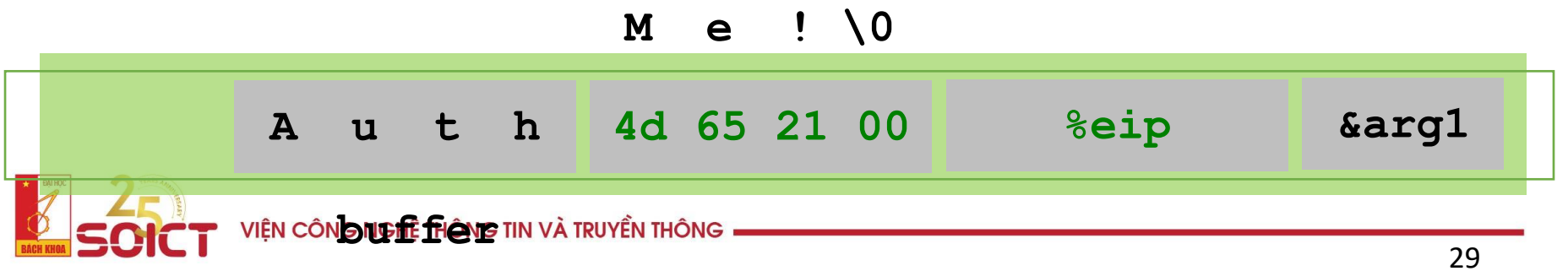


Ví dụ về tràn bộ đệm

```
void func(char *arg1)
{
    char buffer[4];
    strcpy(buffer, arg1);
    return;
}
int main()
{
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```

```
pop %ebp
```

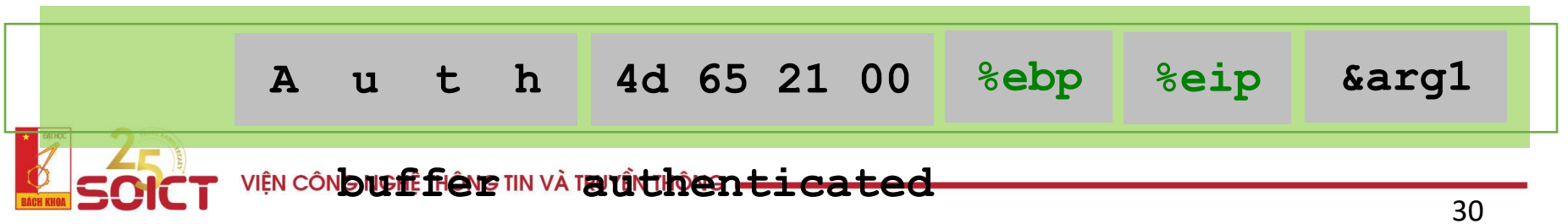
%ebp = 0x0021654d
→SEGMENTATION FAULT



Tràn bộ đệm – Ví dụ khác

```
void func(char *arg1)
{
    int authenticated = 0
    char buffer[4];
    strcpy(buffer, arg1);
    if(authenticated){//privileged execution}
}
int main()
{
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```

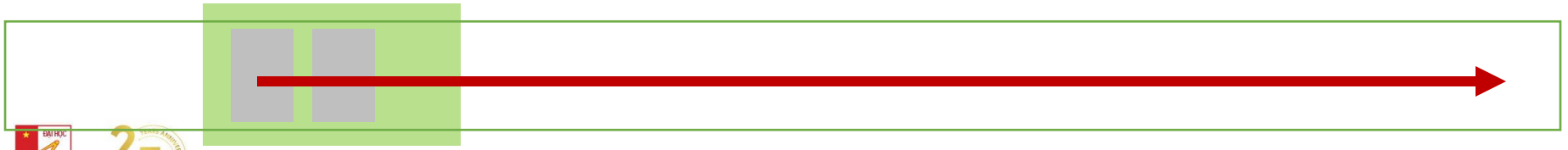
Hàm được thực thi như thế nào?



Tràn bộ đệm – Ví dụ khác

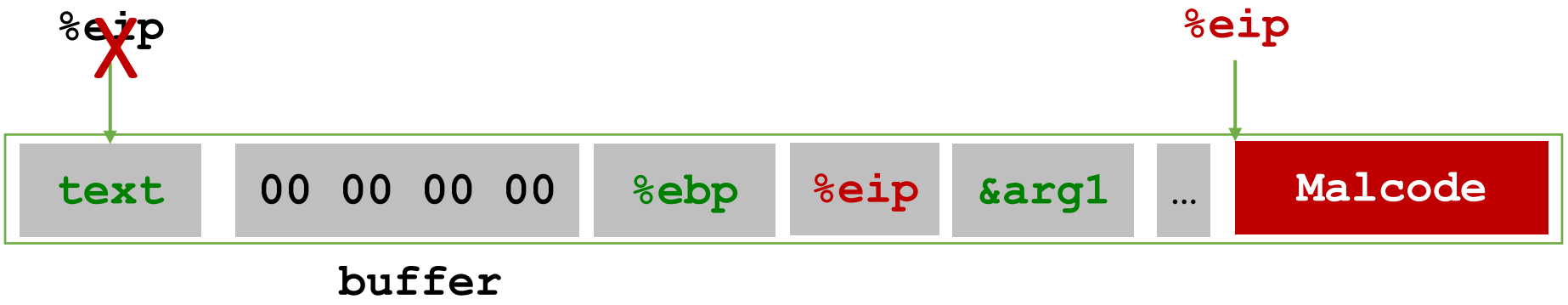
```
void func(char *arg1)
{
    int authenticated = 0
    char buffer[4];
    strcpy(buffer, arg1);
    if(authenticated){//privileged execution}
}
int main()
{
    char *mystr = “AuthMe!”;
    func(mystr);
    ...
}
```

Người dùng có thể ghi đè dữ liệu tùy ý tới các vùng nhớ khác



Khai thác lỗ hổng tràn bộ đệm

- Lỗ hổng tràn bộ đệm cho phép kẻ tấn công truy cập (read/write/execute) tùy ý vào các dữ liệu không xác định
- Phương thức khai thác phổ biến nhất: chèn mã nguồn thực thi (code injection)
- Ý tưởng



Code Injection

- Vấn đề 1: Nạp mã độc(malcode) vào stack
  - Phải là mã máy
  - Không byte có giá trị 0
  - Không sử dụng bộ nạp (loader)
  - Không sử dụng vùng nhớ stack
- Vấn đề 2: Nạp đúng các địa chỉ lệnh thực thi sau khi kết thúc lời gọi hàm → Xác định đúng %eip
  - Mức độ khó khi xác định giá trị %eip phụ thuộc vị trí của malcode
- Vấn đề 3: Nạp đúng địa chỉ trả về → Xác định đúng %ebp

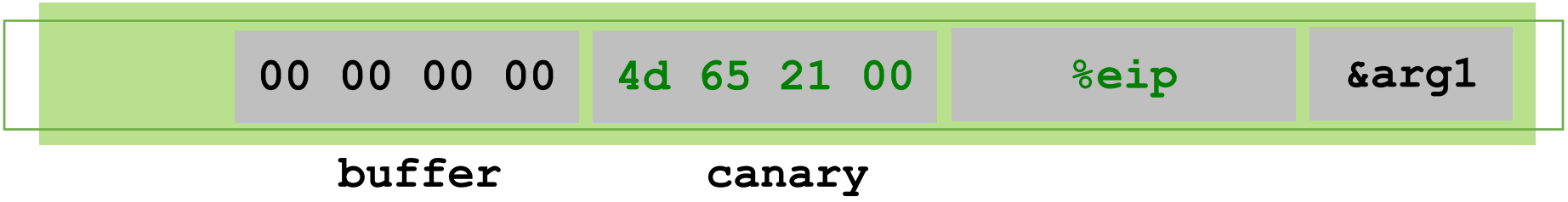
Buffer Overflow – Phòng chống

- **Secure Coding**: sử dụng các hàm an toàn có kiểm soát kích thước dữ liệu đầu vào.
  - fgets(), strncpy(), strncat()…
- Stack Shield:
  - Lưu trữ địa chỉ trả về vào vùng nhớ bảo vệ không thể bị ghi đè
  - Sao chép địa chỉ trả về từ vùng nhớ bảo vệ
- Stack Guard: sử dụng các giá trị canh giữ (canary) để phát hiện mã nguồn bị chèn
- Non-executable stack: Không cho phép thực thi mã nguồn trong stack
  - Linux: sysctl -w kernel.exec-shield=0

Sử dụng giá trị canh giữ - Ví dụ

```
callee()
{
    int canary = random;
    char buffer[];
    ...
    if(canary!=random)
     //detect attack
    else return;
}
```

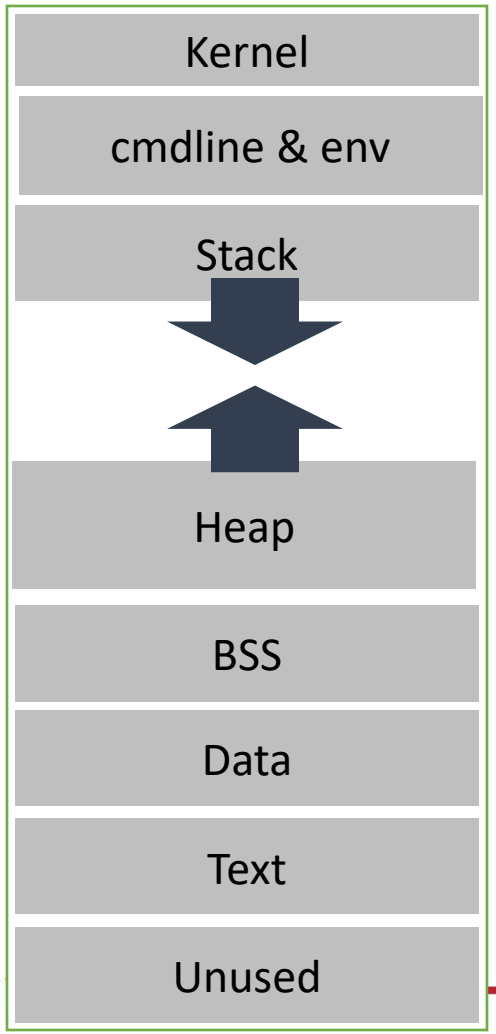
```
static int random;
caller()
{
    random = rand();
    callee();
}
```





Buffer Overflow – Phòng chống

- Address Space Layout Randomization



0xffffffff
0xc0000000

Thiết lập khi tiến trình bắt đầu

Thay đổi khi thực thi

Xác định ở thời điểm biên dịch

Không gian địa chỉ của thiết bị vào-ra

Nạp vào với địa chỉ bắt đầu của mỗi vùng là ngẫu nhiên

0x08048000

0x00000000

Buffer Overflow

- **Secure Coding**: Điều kiện truy cập bộ nhớ khi hàm thực thi
  - Tiền điều kiện(precondition): điều kiện để câu lệnh/hàm được thực thi đúng đắn
  - Hậu điều kiện(postcondition): khẳng định trạng thái đúng đắn của các đối tượng khi lệnh/hàm kết thúc
- Ví dụ: Xác định các điều kiện truy cập bộ nhớ

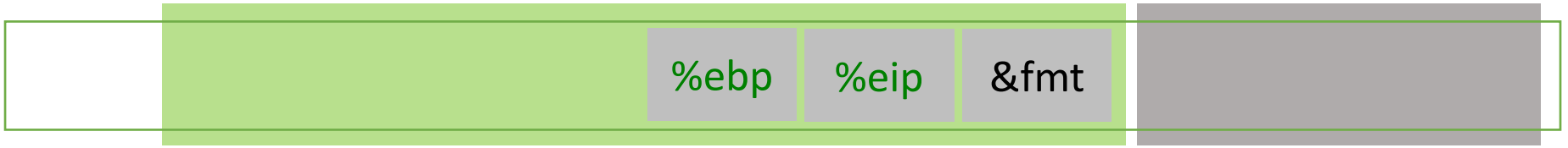
```
void displayArr(int a[], size_t n)
{
    for(size_t i = 0; i < n, i++)
    printf("%d", a[i]);
}
```

3. Một số lỗ hổng truy cập bộ nhớ khác

Lỗ hổng xâu định dạng

- Format String: Xâu định dạng vào ra dữ liệu
- Lỗ hổng Format String: xâu định dạng không phù hợp với danh sách tham số
- Ví dụ

```
void func()
{
    char buf[32];
    if(fgets(buf, sizeof(buf),stdin) == NULL)
     return;
    printf(buf);
}
```



Lỗ hổng xâu định dạng

- printf(“%d”);
  - Hiển thị 4 byte phía trước địa chỉ đầu tiên của stack frame của hàm `printf`
- printf(“%s”);
  - Hiển thị các byte cho tới khi gặp ký tự kết thúc xâu
- printf(“%d%d%d…”)
  - Hiển thị chuỗi byte dưới dạng số nguyên
- printf(“%x%x%x…”)
  - Hiển thị chuỗi byte dưới dạng hexa
- printf(“…%n”):
  - **Ghi** số byte đã hiển thị vào vùng nhớ

Lỗ hổng tràn số nguyên

- Trong máy tính, số nguyên được biểu diễn bằng trục số tròn. Dải biểu diễn:
  - Số nguyên có dấu: $[-2^{n-1}, 2^{n-1} - 1]$
  - Số nguyên không dấu: $[0, 2^n - 1]$
- Integer Overflow: Biến số nguyên của chương trình nhận một giá trị nằm ngoài dải biểu diễn. Ví dụ
  - Số nguyên có dấu: 0x7ff..f + 1 = 0x80..0, 0xff..f + 1 = 0x0
  - Số nguyên không dấu: 0xff..f + 1 = 0x0, 0x0 – 1 = 0xff...f
- Trong phần mềm, việc không kiểm soát hiện tượng tràn số nguyên có thể gây ra lỗ hổng tràn bộ đệm.

Lỗ hổng tràn số nguyên – Ví dụ 1

- Lỗ hổng do ép kiểu số nguyên có dấu sang số nguyên không dấu

```
#define MAX 1024
void vul_func1()
{
    char buff[1024];
    int len = recv_len_from_client();
    char *mess = recv_mess_from_client();
    if (len > 1024)
    printf (“Too large”);
    else
     memcpy(buf, mess, len);
}
```

Lỗ hổng tràn số nguyên – Ví dụ 2

- Lỗ hổng do kết quả phép tính tràn khỏi phạm vi biểu diễn

```
int main()
{
    int *arr;
    int len;
    printf("Number of items: "); scanf("%d", &len);
    arr = malloc(len * sizeof(int));
    for(int i = 0; i < len; i++)
     scanf("%d", arr[i]);
    return 0;
}
```

ĐẠI HỌC BÁCH KHOA

25 YEARS ANNIVERSARY SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



soict.hust.edu.vn/ fb.com/groups/soict